Offline Multi-Floor Capture and Manual Upload: Feature Plan

Nov 5, 2026 · @Sri Varshini

Summary

Engineers should be able to capture many floors in one trip with no network, then review everything and tap **Upload now** once, with no photo lost on the way.

Problem. Today the app starts uploading by itself the moment a network appears, so engineers must preload each floor plan before leaving coverage and cannot open the next floor's plan once they are offline. Photos also go to RAM first because the upload normally starts straight away.

Proposal. Five changes, in this order of dependence:

1. **Offline site pack.** Floor plan images are downloaded and stored on the phone ahead of time, with a visible "available offline" status per floor.
2. **Site visit session with hold mode.** While a session is open, no automatic uploading happens.
3. **Disk-first safe storage.** Each photo is saved to the phone, size-checked and fingerprinted before it counts as captured. Waiting photos are protected from being wiped by login, logout or session expiry.
4. **Upload now flow.** A review screen, pre-checks, then ordered, resumable upload with progress.
5. **Verified deletion.** A photo leaves the phone only after the server confirms it holds an exact copy.

Why now. The current code can delete waiting photos when a user logs in again or the 7-day session expires. That risk exists today and gets worse the longer photos wait, so fixing it is the first deliverable.

How it works today

Photos are saved on the phone, but the app is built to send them immediately, and floor plans are not really stored offline.

Area	What the code does	Where
Waiting photo record	A small JSON record (pin, file name, size, time taken, status) kept in Capacitor Preferences under one key	frontend/src/store/fileUploadQueue.ts
Photo bytes	Held in RAM (max 2), then copied to Directory.Data/pending-uploads in 256 KB chunks and size-checked	fileUploadQueue.ts, safeFileCopy.ts
Pin and capture details	A separate queue in localStorage, replayed in order with retry	frontend/src/store/writeQueue.ts
Upload triggers	Browser online event, Capacitor network change, window focus, login or session restore, app start, and a 20 s poll	fileUploadQueue.ts
Upload order	One pin at a time, oldest photo first; each photo carries its capture time so pin numbers follow shooting order	fileUploadQueue.ts, workflowStore.ts
Retry rules	Unreachable server never counts against the limit; once a server answers, 10 attempts with 2 s to 60 s backoff	fileUploadQueue.ts
Duplicate protection	Server keys each photo by organisation plus SHA-256 of the file	backend/app/services/capture_stitch_service.py
Floor plan image	Loaded from a URL; persisted store keeps floor and room data, not the image	CaptureWorkflowPage.tsx, workflowStore.ts
Session length	Access token 15 minutes, refresh token 7 days	backend/app/core/config.py

After the bytes arrive, a raw Insta360 file returns HTTP 202 with a stitch job. The phone keeps its copy and polls every 4 s until the panorama is ready, then deletes the local file.

Gaps this feature must close

Five gaps stand between today's code and the requirement; the first is a data-loss risk that exists today.

1. **Waiting photos can be wiped.** `clearClientSessionState()` deletes the upload queue and its files (frontend/src/store/sessionIsolation.ts). It runs on every login (LoginPage.tsx), on logout (DashboardLayout.tsx), and when the server confirms there is no valid session (apiClient.ts). The refresh token lasts 7 days, so an engineer offline for a week, or one who logs in again before uploading, can lose every waiting photo.
2. **Floor plans are not stored offline.** The floor plan image is a web link. It appears offline only if the browser happened to cache it, which is why users are told to preload.
3. **Uploads cannot be held.** Every trigger starts a flush automatically. There is no switch for "wait until the user says so".
4. **RAM-first storage assumes a quick upload.** Photos live in memory first and reach disk in the background. For photos that wait hours, disk must be the primary copy, saved and verified before the capture counts.
5. **One JSON blob tracks all waiting photos.** The whole list is rewritten on each change. This is fine for a few photos but fragile for hundreds.

What already helps. Server-side duplicate detection by SHA-256 means a photo sent twice does not become two captures, which makes safe retry much simpler. Capture time already drives pin numbering, so upload order will not scramble numbers.

Design A: offline site pack

Floor plans are saved to the phone before the trip, so any floor can be opened with no network.

- **Download button.** A "Download for offline" action on a project or tower saves every floor plan image into app storage, plus a small manifest listing what is saved and which version.
- **Local first.** The floor plan viewer looks for the saved copy first and falls back to the network only when none exists.

- **Visible status.** Each floor shows "Available offline" or "Needs download". Before the engineer leaves coverage, the app warns, for example "3 floors in Tower B are not downloaded".
- **Stay current.** When online, the app compares plan versions and refreshes a saved plan that changed. Floors sharing one plan image store it once.
- **Space.** Floor plans are small compared with photos, so a whole tower costs little storage.
- **Room and pin data.** The floor, flat and room list is already persisted in the workflow store, so the pack mainly needs to guarantee the images and verify that the data set is complete for the selected scope.

Design B: site visit session and hold mode

A site visit session groups everything captured on one trip, and while it is open nothing uploads on its own.

- **Session.** A container created when the engineer starts a trip. It spans many floors and towers and records who owns it, when it started, and its state (open, ready to upload, uploading, complete).
- **Hold switch.** Every automatic trigger (online event, network change, focus, login restore, 20 s poll, app start) checks the hold flag and does nothing while a session is open. Only **Upload now** releases it.
- **Upload mode setting.** "Upload when I tap" (proposed default for field engineers) or "Auto upload" for users who prefer today's behaviour.
- **Pin and capture details are held too.** Pins and capture records join the session and are sent together with the photos. If pins synced early, managers would see empty pins that look half-finished. The cost is that the server has no partial copy until upload, which is why Design C and E must be strong.
- **Session state is durable.** It survives app restart and shows a "Resume session" prompt on next launch.

Design C: safe on-device storage

A photo only counts as captured once it is on disk, size-checked and fingerprinted, and nothing the user does can silently delete it.

Disk first.

- Save each photo to disk immediately, verify its size, and compute its SHA-256 on the phone at capture time. Keep the fingerprint in the record.
- RAM stays a short-lived helper, not the primary copy.

Better bookkeeping.

- Replace the single JSON blob with a small on-device database (SQLite) or one file per session, so the list of waiting photos is not rewritten in full on every change.

Storage check.

- Photos are about 12 MB each. As an example, 10 floors with 20 pins each is about 2.4 GB.
- Check free space when a session starts, show "space for about N more photos", warn at set thresholds, and never let a save fail silently.

Protection against wipes. This is the first thing to build.

Event	Today	Proposed
Same user logs in again	Queue and files deleted	Waiting photos kept
Different user logs in	Queue and files deleted	Photos stay locked to the original owner; ask what to do, never delete
Logout with unsent photos	Queue and files deleted	Logout blocked, or photos kept with a clear warning
Session expired (server confirms)	Queue and files deleted	Prompt to log in again; keep everything
App cache cleared or app uninstalled	Everything lost	Warn in the app; optional export to shared storage (privacy trade-off to review)

Backup outside the app. Tell engineers not to format the Insta360 card until the upload is confirmed. Storing the camera's original file name with each capture would allow recovery from the card.

Design D: the Upload now flow

Uploading starts only when the user taps **Upload now**, after a review and pre-checks, and it can pause, resume and survive an app restart.

1. **Review screen.** Per floor: photo count, total size, and anything damaged or needing a retake.
2. **Pre-checks.**
    - Is the server reachable?
    - Is the user signed in? If not, prompt a login that wipes nothing.

    - Is the phone on Wi-Fi or mobile data? Warn before sending gigabytes over mobile data.
    - Is there enough battery?
3. **Send in order.** Pins and rooms first, then photos one at a time, oldest first, floor by floor. Keep today's rule of one upload at a time to avoid memory crashes. Capture time, not arrival time, decides pin numbers.
4. **Progress.** For example "Floor 3: 12 of 20, overall 54 of 200", with pause, resume and a per-photo error list. Progress survives an app restart.
5. **Keep the app alive.** Android pauses background work. Phase 1 keeps the screen awake and asks the user to stay in the app. If that proves too limiting, a later phase moves uploading into native Android background work, which is a much larger job.
6. **Failures stay visible.** A failed photo shows Retry or Capture again. Nothing is deleted on failure, consistent with the rule set after the 2026-10-30 field incident.

Design E: proving nothing is lost before deleting

A phone copy is deleted only after the server confirms it holds an exact, matching copy.

- **Per-photo confirmation.** The server returns the SHA-256 it computed. If it differs from the phone's fingerprint, the photo is sent again.
- **Final reconciliation.** After the last photo, the phone sends the server the full list of fingerprints for the session. The server replies with which ones it has; anything missing is re-sent. Only then is the session marked complete.
- **Two states per photo.** "Safely on server" (bytes received and matched) and "processed" (stitching finished). If the server keeps the raw file, the phone can free space at "safely on server" instead of waiting for stitching. This depends on decision 3 below.
- **Safe retries.** Because the server already de-duplicates by fingerprint, re-sending a photo after a dropped connection is harmless.
- **Backend work needed.** Return the computed fingerprint on upload, and add an endpoint that accepts a list of fingerprints for a session and returns which are present.

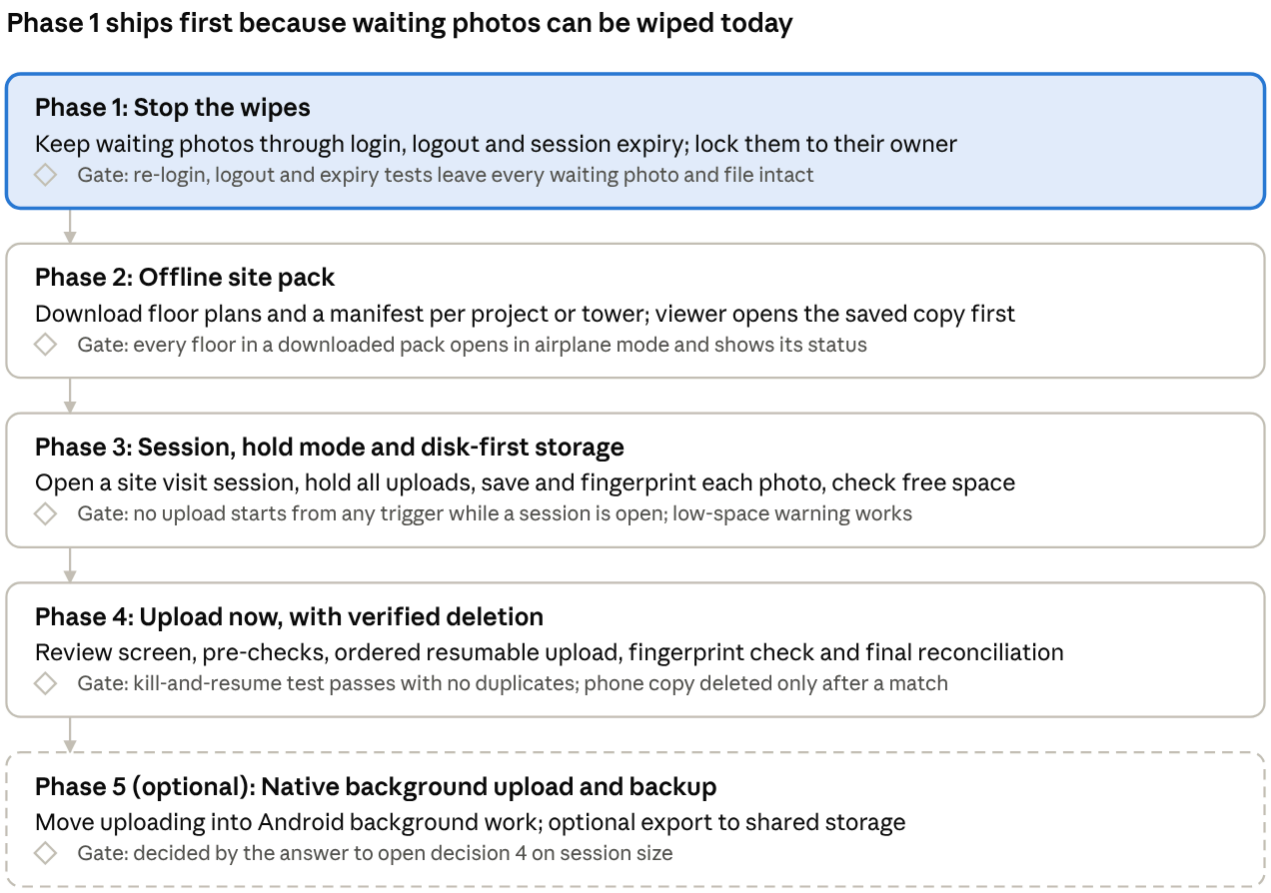
Edge cases and risks

Six risks are not fully solved by the core design and each needs an owner.

Risk	Why it matters	Mitigation
Server stitching backlog	A large session can finish uploading long before panoramas are ready. The server's stitch concurrency has not been checked.	Show "uploaded" and "processed" separately; measure stitch throughput before launch
Floor plan changed while offline	Someone may update a plan or delete pins while the engineer is away	Store the plan version with each pin; server flags a mismatch instead of silently placing pins on the wrong drawing
App cache cleared or app uninstalled	App-private storage is deleted with it	In-app warning; optional export to shared storage; camera card kept until confirmed
Phone runs out of space mid-trip	Photos are about 12 MB each	Free-space check at session start, running "photos remaining" estimate, hard stop with a clear message
App killed during upload	Android may stop background work	Durable progress, automatic resume prompt, keep-awake during upload
Different user on a shared device	Today a new login wipes the previous user's queue	Keep photos locked to the original owner; never auto-delete

Phased delivery plan

Build in five phases, each ending at a gate that must pass before the next begins.



phased roadmap · 5 phases, a gate each

Phases 1 to 4 deliver the requirement; phase 5 is only needed if the session-size answer shows keep-awake is not enough. Phase 4 also needs the two small backend changes listed under Design E. No dates are set because team size and start date are not yet known.

Open decisions

Four decisions change the scope, and none has an owner or date yet.

#	Decision	Recommendation	Why it matters
1	Upload mode: only hold, a setting, or per session	A setting, with "Upload when I tap" as the default for field engineers	Keeps today's behaviour available for users who want it
2	Pin and capture details: held with photos, or synced automatically	Hold them with the photos	Avoids half-finished pins appearing for managers

#	Decision	Recommendation	Why it matters
3	Free phone space at "safely on server" or only after stitching succeeds	Decide after confirming the server keeps the raw file	The first frees space sooner but relies on server retention
4	Realistic session size: floors and pins per trip	Needs your field input	Decides whether keep-awake is enough or native background upload is needed from the start

Acceptance criteria and test plan

The feature is done when an engineer can capture across floors offline, upload once, and every photo is verified on the server with none lost under any of the failure cases below.

Acceptance criteria

- [ ] Every floor plan in a downloaded site pack opens with airplane mode on, and each floor shows its offline status.
- [ ] With a session open, no upload starts from any trigger, including reconnect, focus, login and the 20 s poll.
- [ ] A photo is not reported as captured until it is on disk, size-matched and fingerprinted.
- [ ] Logging in again, logging out, and an expired session never delete waiting photos or their files.
- [ ] **Upload now** sends pins, rooms and photos in capture order, and pin numbers match shooting order.
- [ ] Killing the app mid-upload and reopening resumes without duplicate captures and without re-sending finished photos.
- [ ] A phone copy is deleted only after the server confirms a matching fingerprint and the final reconciliation passes.
- [ ] Low storage produces a clear warning before capture fails, never a silent failure.

Test scenarios

Scenario	Expected result
Capture 3 floors in airplane mode, then upload on Wi-Fi	All photos arrive in order; reconciliation lists none missing

Scenario	Expected result
Log out, then log in as the same user, with photos waiting	Photos still present
Log in as a different user with photos waiting	Photos locked to owner; nothing deleted
Server unreachable partway through upload	Upload pauses; resumes later with no duplicates
App force-killed during a photo write	Capture shown as incomplete and recoverable or re-capturable; nothing silently dropped
Phone storage nearly full	Warning before capture; clear stop message
Photo damaged on disk	Repaired from the camera original if present; otherwise flagged and kept
Large session on a real device	Upload completes without memory crash; measured time recorded